

Faster Lance Index Builds on Kunpeng

Fast assignment · PiPNN partition graphs · ARM SVE/SVE2 kernels

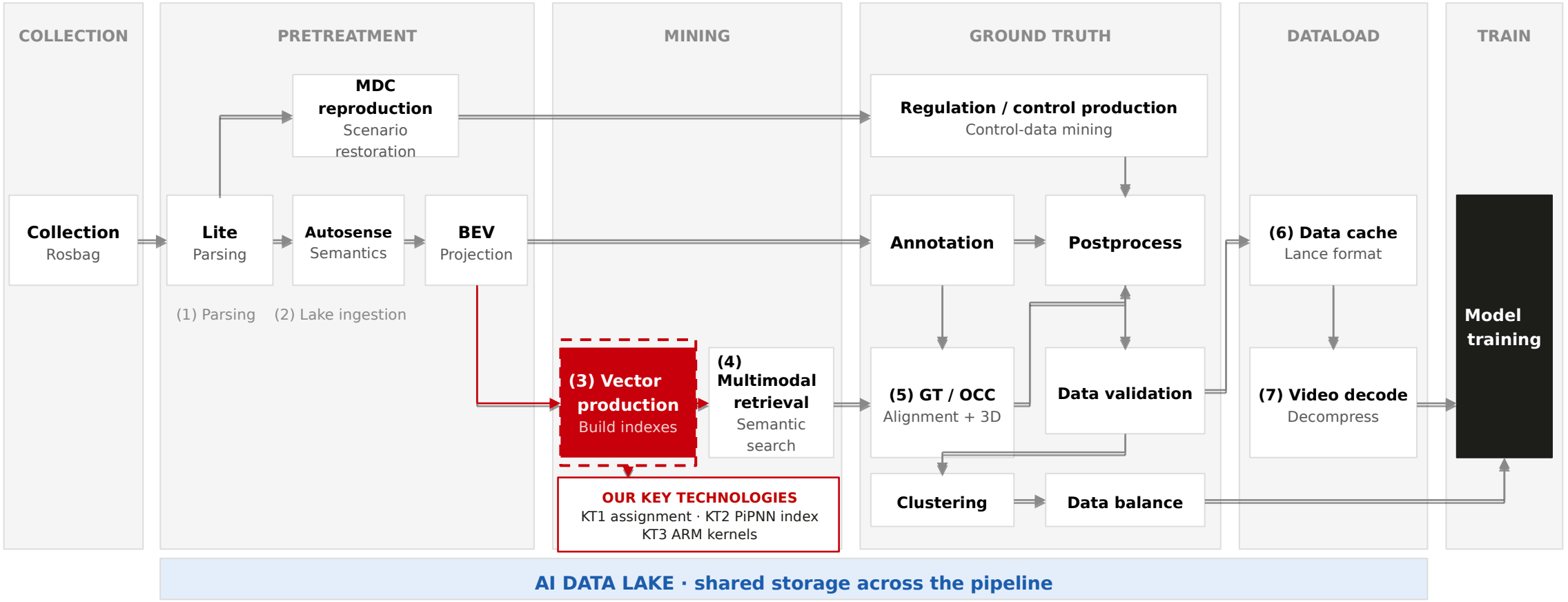
Project proposal v8 · 10 November 2026

Vector Index Optimization

HUAWEI

We speed up ADS stage (3): building vector indexes

[Company context] Redrawn from ADS-PLAN slide 1; target is Vector Production → Building Vector Indexes.

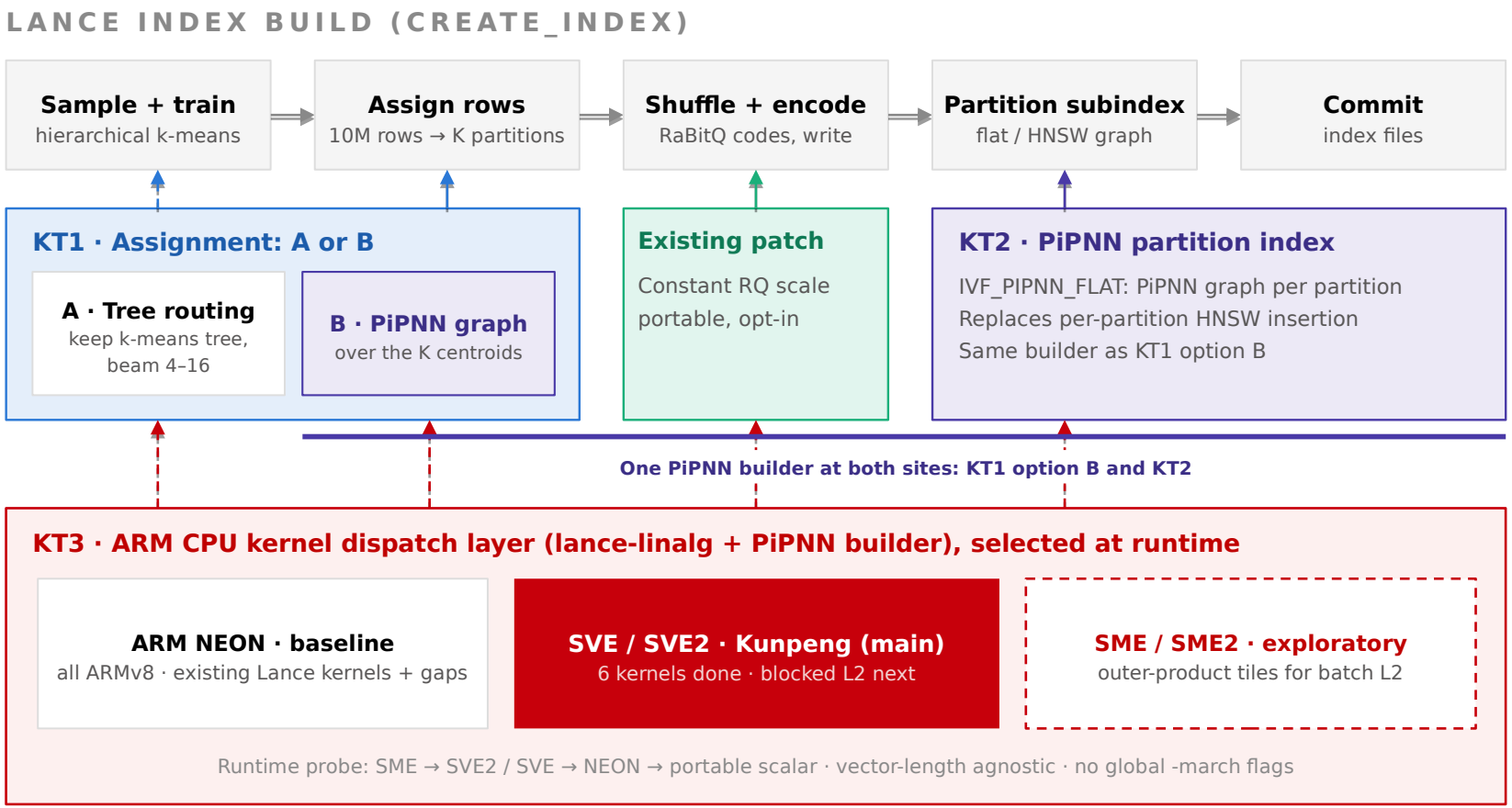


Index build gates retrieval: new vectors wait until it ends. All three key technologies sit in this box.

– Huawei Confidential –

Three key technologies in the Lance build

Lance IVF create_index with each key technology (KT) at its integration point · solid = built and measured in Lance branch · dashed = proposed



STATUS · 10 NOV 2026

KT1 assignment

- A · tree:4: measured, 2.49× faster
- B · PiPNN graph: measured, 2.04× faster

KT2 PiPNN index

- IVF_PIPNN_FLAT builds, persists, searches
- Measured: 4.9× faster, recall ≥ HNSW

KT3 ARM kernels

- 6 SVE/SVE2 kernels, QEMU-validated
- Blocked L2 + argmin: in progress
- Native Kunpeng run: next

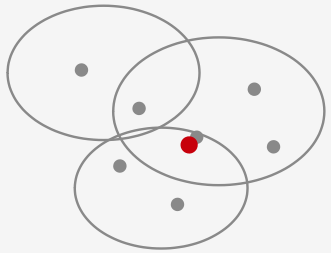
KT1 and KT2 change what is computed; KT3 makes it fast on Kunpeng. Each ships separately.

– Huawei Confidential –

PiPNN · Build the graph with matrix math, not search

Algorithm behind KT1 option B and KT2 · redrawn by us after Rubel et al., KDD 2026 · red tags = where KT3 kernels plug in

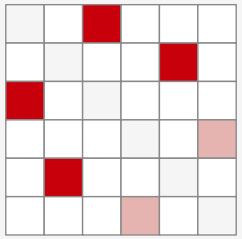
1 · Partition into leaves



Random leaders; each point joins its few nearest → overlapping leaves ≤ 1,024

P0 KERNEL: ROUTING

2 · Leaf Gram → k-NN



All pairs in a leaf as one dense matrix; keep k nearest each way

P0 KERNEL: GRAM

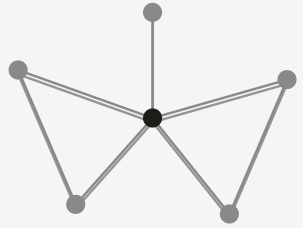
3 · HashPrune

c1 1.2
c2 2.0
c3 1.5
c4 3.1 evicted
c5 0.9

Bounded reservoir per point keeps near candidates in varied directions

P1: SKETCHES

4 · RobustPrune



Final degree-bounded graph; search starts at the medoid

P1: BATCHED L2

HNSW today · insert one vector at a time

· each insert runs a graph search, then links + prunes
· pointer chasing, locks, serial dependency

PiPNN · no search during build

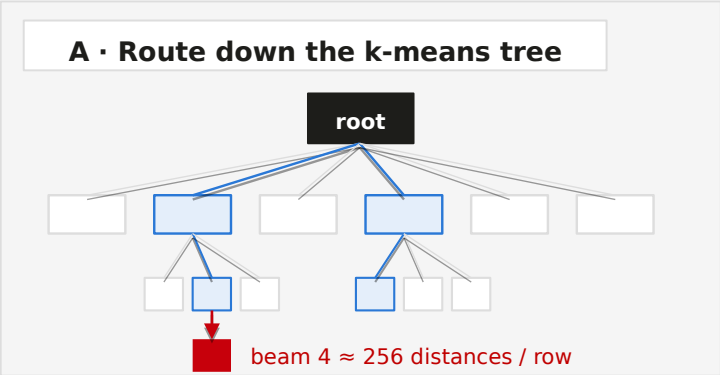
· leaves are independent dense blocks: parallel, SIMD-friendly
· used twice: graph over centroids (KT1-B) and per partition (KT2)

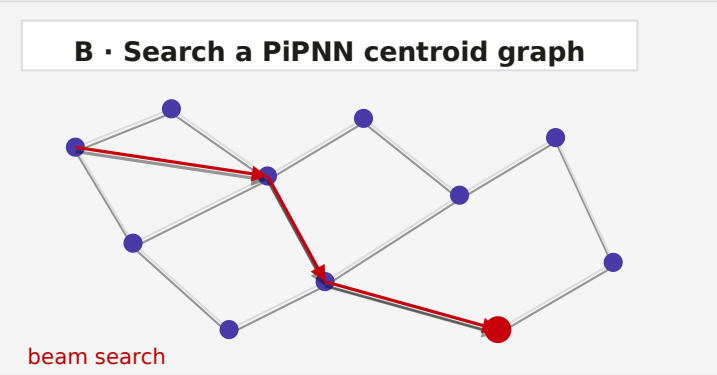
PiPNN turns graph building into kernel work, which is why it depends on KT3 on Kunpeng.

– Huawei Confidential –

KT1 · Tree or PiPNN assignment: ~2× faster build

[x86] BIGANN10M (10M × 128), IVF_RQ 5-bit, K = 16,384, same centroids (only assignment differs) · AMD EPYC 9124 · 1 run each (±1 s, ±0.005 noise)





Headline

2.49×
A · tree:4 vs exact

2.04×
B · PiPNN beam 15

· recall and p50/p90 unchanged (±1 ms)
· no gain at small K (GIST1M, K = 1,000)

RESULTS · BIGANN10M · RECALL@10 AND LATENCY AT NPROBES 256, 1,970 QUERIES

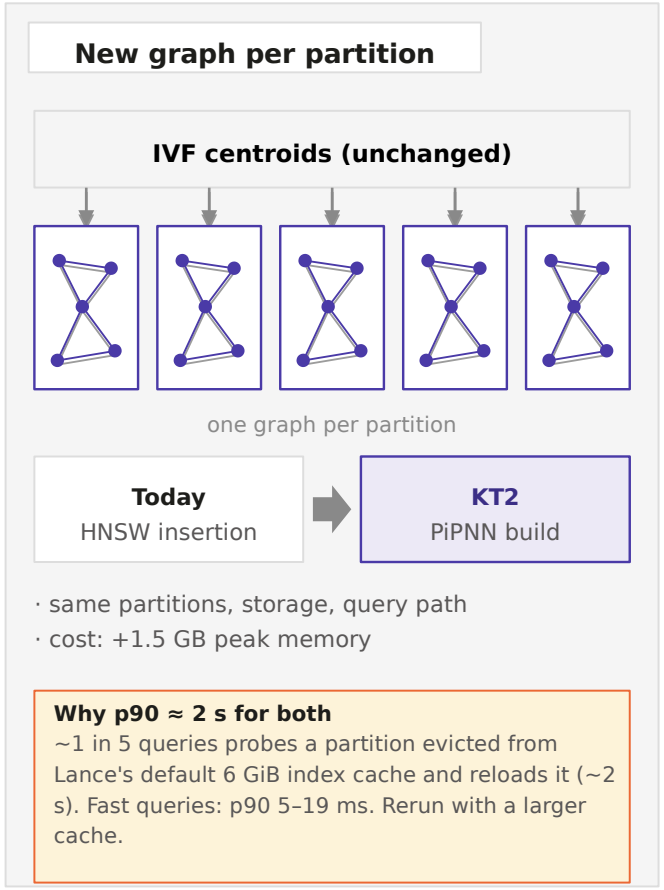
Method	Build s	Speedup	Assign CPU s	Recall@10	p50 ms	p90 ms
exact (baseline)	37.67	1.00×	760.9	0.9430	9.63	9.93
Lance default (HNSW)	27.07	1.39×	376.0	0.9423	9.06	9.43
A · tree:4	**15.11**	**2.49×**	**59.4**	**0.9442**	**9.30**	**9.63**
A · tree:16	18.97	1.99×	137.3	0.9443	9.78	10.16
B · PiPNN beam 15	**18.43**	**2.04×**	**174.2**	**0.9417**	**9.23**	**9.50**
B · PiPNN beam 32	19.98	1.89×	221.8	0.9428	9.06	9.34

Both options halve build time at the same recall and latency; tree:4 is fastest, PiPNN beam 15 close behind.

– Huawei Confidential –

KT2 · PiPNN partition graphs build 4.9× faster

[x86] IVF_PIPNN_FLAT vs IVF_HNSW_FLAT, BIGANN10M (10M × 128 f32), K = 5, same exact assignment · AMD EPYC 9124 · 1 build each, separate processes



Build cost

Index	Build wall s	Build CPU s	Index size MB	Peak RSS GB
HNSW	537.7	12,590	6,994	16.2
PiPNN	109.9	1,830	5,915	17.7
Change	**4.9× faster**	**6.9× less**	**−15%**	+1.5 GB

Recall@10 and latency (ms) · 1,970 queries

nprobes · ef	R@10 HNSW	R@10 PiPNN	p50 HNSW	p50 PiPNN	p90 HNSW	p90 PiPNN
1 · 32	0.744	**0.767**	4.4	3.3	2,004	2,144
1 · 64	0.802	**0.821**	4.2	4.1	1,972	2,181
1 · 128	0.839	**0.848**	7.0	6.3	1,982	1,997
1 · 256	0.855	**0.857**	5.3	7.5	1,995	2,014
1 · 512	0.860	0.859	5.9	7.0	1,997	2,163
2 · 32	0.845	**0.874**	6.1	5.5	2,138	3,028
2 · 64	0.909	**0.935**	10.0	10.5	2,145	3,066
2 · 128	0.949	**0.964**	12.2	11.9	2,136	3,209
2 · 256	0.966	**0.973**	12.9	9.9	2,319	2,580
2 · 512	0.971	**0.976**	10.2	13.8	2,097	2,591

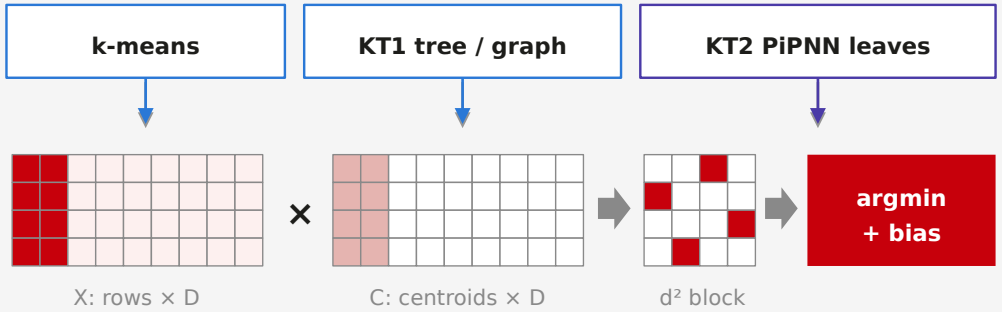
Drop-in for IVF_HNSW_FLAT: faster, smaller, as accurate. Memory is the trade-off.

– Huawei Confidential –

KT3 · One blocked ARM kernel under KT1 and KT2

[Proposal] diagram drawn by us, sizes illustrative · SVE is vector-length agnostic · correctness checked in QEMU, Kunpeng timings pending

Blocked L2 + argmin: rows × centroids



· Today: pairwise NEON, one vector × all centroids
· Blocked: each loaded chunk feeds 4×4 dot products in registers
· Predicated tails: same code at any vector length

Selected at runtime on ARM

NEON	SVE / SVE2	SME / SME2
baseline	Kunpeng · main	exploratory

Kernel status

ARM kernel	Serves	Status	Kunpeng
f32 dot + L2, pair and batch	flat / HNSW search, k-means	done · QEMU ✓	**TBD**
u8 dot (UDOT) + u8 L2	uint8 vectors (BIGANN)	done · QEMU ✓	**TBD**
4-bit LUT accumulate	RQ / PQ fast scan	done · QEMU ✓	**TBD**
P0 · blocked L2 + argmin / top-k	k-means, KT1, PiPNN routing	in progress	**TBD**
P0 · leaf Gram + top-k	KT2 PiPNN leaf k-NN	planned	**TBD**

QEMU ✓ = clang-18 -Werror, validated at 5 vector lengths (128–2048-bit), VL changed mid-process, guard pages for tail over-reads.

First gate: blocked beats pairwise at equal vector width on the same Kunpeng host, with recall within ±0.005. If not, the dispatcher keeps the old path.

Kernel speed on Kunpeng is the open question; KT3 answers it for both KT1 and KT2.

– Huawei Confidential –

Summary · measured on x86, Kunpeng next

Three key technologies · x86 results measured by us · [Proposal] sizes S ≈ 1–2, M ≈ 3–5, L ≈ 6–10 engineer-weeks

KT1 · Assignment

2.5×
faster build [x86]

· A tree:4 · B PiPNN 2.0×
· recall within 0.005

KT2 · PiPNN index

4.9×
faster build [x86]

· 6.9× less CPU, −15% size
· recall ≥ HNSW at 9/10

KT3 · ARM kernels

TBD
Kunpeng speedup

· 6 SVE kernels done
· blocked L2 + Gram: next

ROADMAP · EACH PHASE HAS A GATE

1 · Kunpeng bring-up · S → native build, SVE vs NEON baseline

2 · P0 SVE kernels (KT3) · M → blocked L2 + argmin, leaf Gram

3 · Integrate on ARM · M → KT1 + KT2 with new kernels

4 · Harden, upstream · L → repeats, full grids, Lance PRs

Nothing is forced on: every kernel sits behind runtime dispatch and every algorithm is an opt-in build option, so a failed gate keeps the current path.

Two key technologies measured on x86; kernel speed on Kunpeng is next. Phase 1 fills every TBD.

– Huawei Confidential –